KIM SANGYOON

1171010435

Part A

Implementation and design:

We are trying to fork a process and make child and parent process. In order to do this, I first designed the process of child process when pid of it is equal to 0 and designed the process of parent process when pid of it is greater than 0. Then for each process I printed pid for each of them. After parent prints out its pid, parent process waits for the signal from the child process by using waitpid. This makes child process to proceed by executing the test files like abort and stop. After the child process executes the test files, it will send signals such as abort and stop to the parent process. After parent process receives signal, it will print out normal termination with exit status = 0, and if it is signaled by core dump or termination, it will search the signal number from the signal function that I made and print out the signal that the parent process received. And for the stopped signal, it will receive stop signal and print out stop signal. To add, in order to make parent receive stop signal from child, I used waitpid(pid, &status, WUNTRACED), which reports stopped, terminated, and core dumped signals.

Execution:

In the directory of /csc3150/Assignment_1_117010435/source/program1, we compile the files and to execute, we type ./program1 ./(filename)

Output:

```
root@ubuntu-xenial:/home/vagrant/csc3150/Assignment_1_117010435/source/program1# ./program1 ./abort
Process start to fork
I'm the Parent Process, my pid = 2351
I'm the Child Process, my pid = 2352
Child process start to execute test program:
------------CHILD PROCESS START------------
This is the SIGABRT program

Parent process receives SIGCHLD signal
child process get SIGABRT signal
root@ubuntu-xenial:/home/vagrant/csc3150/Assignment_1_117010435/source/program1# ./program1 ./alarm
Process start to fork
I'm the Parent Process, my pid = 2396
I'm the Child Process, my pid = 2397
Child process start to execute test program:
------------CHILD PROCESS START------------
This is the SIGALRM program

Parent process receives SIGCHLD signal
child process get SIGALRM signal
root@ubuntu-xenial:/home/vagrant/csc3150/Assignment_1_117010435/source/program1# ./program1 ./bus
Process start to fork
I'm the Parent Process, my pid = 2460
I'm the Child Process, my pid = 2461
Child process start to execute test program:
------------CHILD PROCESS START------------
This is the SIGBUS program

Parent process receives SIGCHLD signal
child process get SIGBUS signal
root@ubuntu-xenial:/home/vagrant/csc3150/Assignment_1_117010435/source/program1# ./program1 ./floating
Process start to fork
I'm the Parent Process, my pid = 2517
I'm the Child Process, my pid = 2518
Child process start to execute test program:
------------CHILD PROCESS START------------
This is the SIGFPE program

Parent process receives SIGCHLD signal
child process get SIGFPE signal
```

```
root@ubuntu-xenial:/home/vagrant/csc3150/Assignment_1_117010435/source/program1# ./program1 ./hangup
Process start to fork
I'm the Parent Process, my pid = 2566
I'm the Child Process, my pid = 2567
Child process start to execute test program:
------------CHILD PROCESS START------------
This is the SIGHUP program

Parent process receives SIGCHLD signal
child process get SIGHUP signal
root@ubuntu-xenial:/home/vagrant/csc3150/Assignment_1_117010435/source/program1# ./program1 ./illegal_instr
Process start to fork
I'm the Parent Process, my pid = 2631
I'm the Child Process, my pid = 2632
Child process start to execute test program:
------------CHILD PROCESS START------------
This is the SIGILL program

Parent process receives SIGCHLD signal
child process get SIGILL signal
root@ubuntu-xenial:/home/vagrant/csc3150/Assignment_1_117010435/source/program1# ./program1 ./interrupt
Process start to fork
I'm the Parent Process, my pid = 2680
I'm the Child Process, my pid = 2681
Child process start to execute test program:
------------CHILD PROCESS START------------
This is the SIGINT program

Parent process receives SIGCHLD signal
child process get SIGINT signal
root@ubuntu-xenial:/home/vagrant/csc3150/Assignment_1_117010435/source/program1# ./program1 ./kill
Process start to fork
I'm the Parent Process, my pid = 2745
I'm the Child Process, my pid = 2746
Child process start to execute test program:
------------CHILD PROCESS START------------
This is the SIGKILL program

Parent process receives SIGCHLD signal
child process get SIGKILL signal
```

```
root@ubuntu-xenial:/home/vagrant/csc3150/Assignment_1_117010435/source/program1# ./program1 ./normal
Process start to fork
I'm the Parent Process, my pid = 2768
I'm the Child Process, my pid = 2769
Child process start to execute test program:
------------CHILD PROCESS START------------
This is the normal program

------------CHILD PROCESS END------------
Parent process receives SIGCHLD signal
Normal termination with EXIT STATUS = 0
root@ubuntu-xenial:/home/vagrant/csc3150/Assignment_1_117010435/source/program1# ./program1 ./pipe
Process start to fork
I'm the Parent Process, my pid = 2812
I'm the Child Process, my pid = 2813
Child process start to execute test program:
------------CHILD PROCESS START------------
This is the SIGPIPE program

Parent process receives SIGCHLD signal
child process get SIGPIPE signal
root@ubuntu-xenial:/home/vagrant/csc3150/Assignment_1_117010435/source/program1# ./program1 ./quit
Process start to fork
I'm the Parent Process, my pid = 2835
I'm the Child Process, my pid = 2836
Child process start to execute test program:
------------CHILD PROCESS START------------
This is the SIGQUIT program

Parent process receives SIGCHLD signal
child process get SIGQUIT signal
root@ubuntu-xenial:/home/vagrant/csc3150/Assignment_1_117010435/source/program1# ./program1 ./segment_fault
Process start to fork
I'm the Parent Process, my pid = 2861
I'm the Child Process, my pid = 2862
Child process start to execute test program:
------------CHILD PROCESS START------------
This is the SIGSEGV program

Parent process receives SIGCHLD signal
child process get SIGSEVG signal
```

```
root@ubuntu-xenial:/home/vagrant/csc3150/Assignment_1_117010435/source/program1# ./program1 ./stop
Process start to fork
I'm the Parent Process, my pid = 2889
I'm the Child Process, my pid = 2890
Child process start to execute test program:
------------CHILD PROCESS START------------
This is the SIGSTOP program

Parent process receives SIGCHLD signal
child process get SIGSTOP signal
root@ubuntu-xenial:/home/vagrant/csc3150/Assignment_1_117010435/source/program1# ./program1 ./trap
Process start to fork
I'm the Parent Process, my pid = 2933
I'm the Child Process, my pid = 2934
Child process start to execute test program:
------------CHILD PROCESS START------------
This is the SIGTRAP program

Parent process receives SIGCHLD signal
child process get SIGTRAP signal
root@ubuntu-xenial:/home/vagrant/csc3150/Assignment_1_117010435/source/program1# ./program1 ./trap
Process start to fork
I'm the Parent Process, my pid = 2961
I'm the Child Process, my pid = 2962
Child process start to execute test program:
------------CHILD PROCESS START------------
This is the SIGTRAP program

Parent process receives SIGCHLD signal
child process get SIGTRAP signal
```

Part B

Implementation and design:

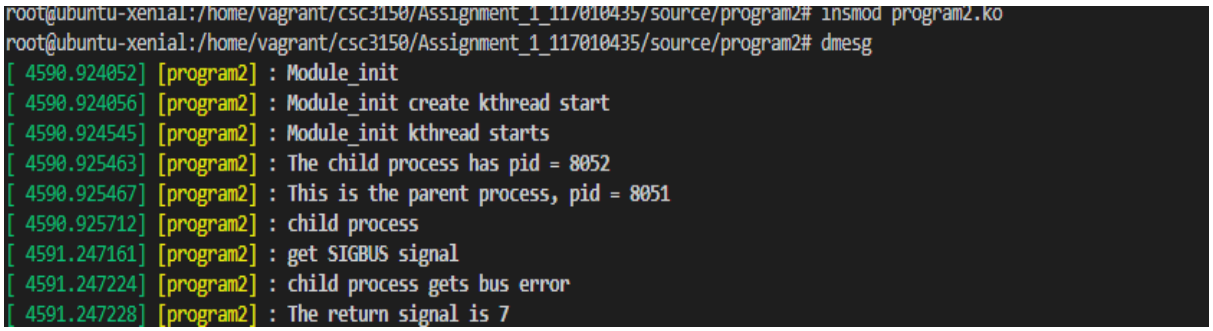
First the kernel module is initialized. Then we have to create thread and fork process. In order to do this, we create a kernel thread and execute my_fork, fork a process in my_fork, and execute the test file. Then in the kernel thread, parent process waits child's signal and after parent receives the signal, it starts printing the received signal. I first defined WTERMSIG, WIFEXITED, WIFESIGNALLED, WIFESTOPPED, which are from sys/wait.h. Then I declared extern functions in order to use these functions exported from linux modules. Then I wrote my_exec, signal, my_wait, my_fork functions. my_exec is the function to get the test file, signal function is to print the signal and return the signal value, my_wait function is to retrieve the signal from the the child process, my_fork function is to create a child process and we execute child process in do_execve().

To add, in order to use functions from kernel modules, we have to export the symbols from the module and mae bzImage, make module install and install. Then we should use extern in program2.c.

Execution:

In the /csc3150/Assignment_1_117010435/source/program2 directory, we have to first compile program2, then insmod program2.ko to insert the module, then dmesg to see the result. Then we should make sure to rmmod and I did dmesg –clear to clear the previous logs.

Output:



Outcome:

From this assignment, I learned how to use kernel modules and functions from it by using export symbol and declaring extern in our main file like program2.c. Learned how to use fork and separately execute the process on child and parent process.